Teoría de Bases de Datos — Laboratorio 4: SQL sobre el motor

Licenciatura en Ciencias de la Computación · FCEIA–UNR

Material. Tres archivos, con una sola fuente de verdad:

- `canon-postgresql.sql` — crea `bd_curso` y la carga *completa*. Es el que se usa en toda la práctica y en este laboratorio. Ejecutarlo de nuevo restaura el estado inicial en cualquier momento: es la herramienta de recuperación ante cualquier estado inesperado.
- `canon-postgresql-nucleo.sql` — sólo la instancia mínima, la que dibujan las diapositivas. Cargado solo, reproduce al pie de la letra las tablas de resultado de la teoría.
- `canon-postgresql-extension.sql` — las filas restantes, para agregar sobre el núcleo.

Los dos últimos se generan desde el primero, así que no se editan: se vuelven a generar.

Ejercicio 1. Puesta en marcha

a) Conectarse al servidor con el cliente de línea de comandos y ejecutar el script:

```
createdb bd_curso
psql -d bd_curso -f canon-postgresql.sql
```

b) Verificar el estado inicial con la siguiente consulta y anotar los seis valores:

```
SELECT (SELECT COUNT(*) FROM persona)    AS personas,
       (SELECT COUNT(*) FROM telefono)   AS telefonos,
       (SELECT COUNT(*) FROM estudiante) AS estudiantes,
       (SELECT COUNT(*) FROM docente)    AS docentes,
       (SELECT COUNT(*) FROM materia)    AS materias,
       (SELECT COUNT(*) FROM cursa)      AS cursados;
```

c) Ejecutar `\i canon-postgresql.sql` otra vez y repetir la verificación: los valores deben ser idénticos. ¿Por qué el script puede ejecutarse muchas veces sin acumular filas duplicadas? Buscar la respuesta en el propio archivo.

Ejercicio 2. Del papel al motor

a) Ejecutar las sentencias propias de los ejercicios 1 a 6 de la práctica y comparar cada resultado con lo resuelto en papel (restaurando el estado inicial donde corresponda). Registrar las diferencias: cada una es un error en la sentencia, un error en la resolución en papel, o una lectura equivocada del enunciado.

b) Para las sentencias que fallan, identificar en el mensaje del motor el número de error y qué parte del texto nombra la restricción violada. Comprobar con un `SELECT` que la base de datos quedó exactamente como estaba: el motor rechaza la sentencia completa, no la aplica a medias.

c) Ejecutar la consulta del ítem 2.d) de la práctica sin el `DISTINCT`. ¿Cuántas filas devuelve y por qué exactamente esa cantidad? Identificar a qué cursado corresponde cada fila repetida.

d) En la consulta del ítem 3.b), reemplazar el `LEFT JOIN` por un `JOIN` común. ¿Qué fila del resultado desaparece y por qué?

Ejercicio 3. Acciones referenciales con ALTER TABLE

En el esquema de `canon-postgresql.sql` ninguna clave foránea declara una acción referencial. Este ejercicio modifica el esquema para observar el otro comportamiento.

a) Intentar eliminar de `estudiante` a Ibáñez (dni 43978415) y anotar el error.

b) Consultar los nombres de las claves foráneas de `cursa`:

```
SELECT conname, confrelid::regclass AS tabla_referenciada
FROM pg_constraint
WHERE conrelid = 'cursa'::regclass AND contype = 'f';
```

c) Reemplazar la clave foránea hacia `estudiante` por una idéntica con `ON DELETE CASCADE`:

```
ALTER TABLE cursa DROP CONSTRAINT fk_cursa_estudiante;
ALTER TABLE cursa ADD CONSTRAINT fk_cursa_estudiante
  FOREIGN KEY (dni) REFERENCES estudiante (dni)
  ON DELETE CASCADE;
```

d) Repetir la eliminación del ítem a). Verificar cuántas filas quedaron en `cursa` y comprobar con un `SELECT` que la fila de Ibáñez en `persona` sigue existiendo. Comparar con lo respondido en el ítem 6.h) de la práctica.

e) Restaurar el estado inicial con `canon-postgresql.sql`. Escribir el `ALTER TABLE` análogo para que la clave foránea de `cursa` hacia `materia` tenga `ON UPDATE CASCADE`, cambiar el código TBD por BD02 en `materia`, y verificar cuántas filas de `cursa` cambiaron. Si el `UPDATE` falla, ¿qué otra tabla lo impide y qué hace falta para que el cambio se aplique también ahí? Comparar con el ítem 6.i) de la práctica.

f) Restaurar el estado inicial. Verificar con la consulta del ítem b) que las claves foráneas volvieron a quedar sin acción referencial declarada.

Ejercicio 4. Programación en la base de datos

En el ejercicio 7 de la práctica usted anticipó en papel qué iba a hacer un trigger y cómo reescribir un procedimiento. Acá se comprueba.

4.1 El trigger, funcionando

a) Restaurar el estado inicial y crear la tabla de auditoría y el trigger del ítem 7.1 de la práctica:

```
CREATE TABLE auditoria_cursado (
  id     integer   GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  dni    integer   NOT NULL,
  codigo varchar(4)   NOT NULL,
  nota   smallint,
  cuando timestamp NOT NULL);

CREATE FUNCTION auditar_cursado() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO auditoria_cursado (dni, codigo, nota, cuando)
  VALUES (NEW.dni, NEW.codigo, NEW.nota, now());
  RETURN NEW;
END; $$;

CREATE TRIGGER cursado_alta
AFTER INSERT ON cursa
FOR EACH ROW EXECUTE FUNCTION auditar_cursado();
```

b) Ejecutar las cuatro sentencias del ítem 7.1, una por vez, contando las filas de `auditoria_cursado` después de cada una. Comparar con lo que había anotado en papel. Cada diferencia hay que explicarla.

c) La cuarta sentencia falla. Anotar el número de error y comprobar con un `SELECT` que la auditoría no registró nada. ¿Qué garantiza eso sobre el trigger y la sentencia que lo dispara?

d) Escribir el segundo trigger que hace falta para que los cambios de nota también queden auditados. Probarlo con un `UPDATE` y verificar.

e) Ver los triggers declarados con `\dS cursa` y encontrar en la salida qué evento y qué momento tiene cada uno.

4.2 La lógica invisible

a) Abrir una sesión nueva, como si fuera otra aplicación que no conoce este esquema. Ejecutar un `INSERT` común en `cursa` y después `SELECT COUNT(*) FROM auditoria_cursado;`

b) La aplicación ejecutó una sentencia y pasaron dos cosas. Nada en el texto del `INSERT` lo anticipa. Explicar dónde tendría que mirar un programador que no escribió este esquema para enterarse.

c) Suponga que la auditoría empieza a fallar en producción. La aplicación no cambió y nadie tocó su código. Enumerar los lugares donde habría que buscar la causa.

4.3 El cursor heredado

a) Restaurar el estado inicial y crear el procedimiento con cursor del ítem 7.3 de la práctica, tal cual está escrito ahí.

b) Anotar los promedios actuales de los doce estudiantes. Ejecutarlo con `CALL recalcular_promedios();` y anotar los promedios nuevos: sobre la instancia sin tocar, ninguno cambia.

c) Cargar una nota nueva —por ejemplo, un 2 de López (44876052) en `AL01`— y volver a ejecutar el procedimiento. Anotar qué promedios cambiaron y cuáles no.

d) Restaurar el estado inicial y escribir **su** versión: la misma tarea en una sola sentencia, sin cursor ni bucle, como la resolvió en papel. Ejecutarla.

e) Comparar las dos salidas estudiante por estudiante. Deben ser idénticas; si no lo son, la diferencia está en el tratamiento de los estudiantes sin cursados.

f) Verificar el caso límite: ¿qué promedio quedó para los dos estudiantes que no cursaron ninguna materia? Confrontar con lo discutido en el ítem 7.2 de la práctica.

g) Su versión y la del cursor consultan `cursa` una cantidad distinta de veces, que usted contó en el ítem 7.3.c). Con doce estudiantes la diferencia todavía no se siente. En el laboratorio de la Unidad 7 esta misma comparación se repite sobre una base de datos de cincuenta mil estudiantes, y ahí se mide.

4.4 Dejar todo como estaba

a) Ejecutar `canon-postgresql.sql` y comprobar con `\dt` que `auditoria_cursado` sigue existiendo. Explicar por qué restaurar la base de datos no la eliminó, y qué alcance tiene un script de creación.

b) Eliminar los triggers, sus funciones, el procedimiento y la tabla de auditoría:

```
DROP TRIGGER IF EXISTS cursado_cambio_nota ON cursa;
DROP TRIGGER IF EXISTS cursado_alta ON cursa;
```

```
DROP FUNCTION IF EXISTS auditar_cambio_nota();
DROP FUNCTION IF EXISTS auditar_cursado();
DROP PROCEDURE IF EXISTS recalcular_promedios;
DROP TABLE IF EXISTS auditoria_cursado;
```

c) Ahora sí, con todo eliminado, restaurar el estado inicial y verificar con la consulta del ejercicio 1.b) que los seis valores volvieron a ser los de partida.

d) `DROP TABLE` eliminó la tabla de auditoría junto con sus datos, sin preguntar. Contrastar con lo que ocurrió en el ejercicio 3 al intentar borrar una fila referenciada. ¿Por qué el motor cuida una cosa y no la otra?

Ejercicio 5. La misma consulta, dos instancias

Durante todo el laboratorio trabajaron con los datos completos. Las diapositivas de la teoría, en cambio, muestran los resultados sobre el *núcleo*: la instancia mínima, la que está dibujada. Acá se ve qué significa esa diferencia.

a) Cargar el núcleo, que es una carga limpia y deja atrás todo lo creado en los ejercicios anteriores:

```
\i canon-postgresql-nucleo.sql
SELECT COUNT(*) FROM cursa;
SELECT p.nombre, c.nota
FROM   persona p JOIN cursa c ON p.dni = c.dni;
```

Comparar el último resultado con el que muestra la diapositiva 36 de la teoría. Debe coincidir fila por fila, incluido el orden.

b) Cargar la extensión sin volver a crear la base de datos, y repetir *exactamente* las mismas dos consultas:

```
\i canon-postgresql-extension.sql
```

Anotar cuántas filas devuelve ahora cada una.

c) El texto de la consulta no cambió ni una letra y el resultado sí. Decir con precisión qué cambió, y por qué entonces es incorrecto afirmar que una consulta "da" un resultado.

d) Volver a mirar el segundo `SELECT` del punto 1. Con el núcleo aparecen cuatro filas y hay personas que no figuran, aunque existen en `persona`. Con la extensión cargada siguen sin figurar. Identificarlas y explicar por qué ninguna de las dos instancias las muestra — la razón no está en los datos sino en la consulta.

e) Restaurar los datos completos con `\i canon-postgresql.sql` y verificar con la consulta del ejercicio 1.b) que los seis valores son los de partida.